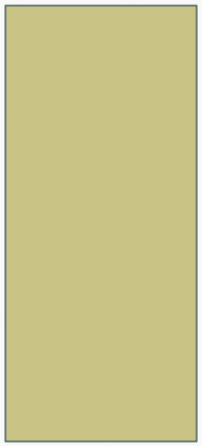
DATA STRUCTURES

LECTURE 03



BOOK READINGS

- C++ How to program , Deitel & Deitel Chapter 7

TEMPLATES

- Generic Programming (one template->multiple data types)
- 2 types of templates
  - Function templates
  - Class templates

FUNCTION TEMPLATES

- How to deal with different data types using functions
- Without templates
  - We rewrite (overload) functions Min(), Max(), and InsertionSort() for many different types
  - There has to be a better way (where we do not have to rewrite)
- Function template
  - Describes a function format that when instantiated with particular data types generates a function definition
    - Write once, use multiple times
    - Compact way to make overloaded functions

FUNCTION TEMPLATE EXAMPLE : MIN

Indicates a template is being defined

Indicates T is our formal template parameter

```
template <class T>
  T Min(T a, T b) {
    if (a < b)
        return a;
    else
        return b;
  }
```

Instantiated functions will return a value whose type is the actual template parameter

Instantiated functions require two actual parameters of the same type. Their type will be the actual value for T

MIN TEMPLATE

- Code segment (e.g. in main() function)

```
int Input1, Input2; cin>>Input1>>Input2;
cout << Min(Input1, Input2) << endl;
```

- Causes the following function to be generated from our template

```
int Min(int a, int b) {
  if (a < b)
      return a;
  else
      return b;
}
```

MIN TEMPLATE

- Code segment (e.g. in main() function)

```
double Value1 = 4.30;
double Value2 = 19.54;
cout << Min(Value1, Value2) << endl;
```

- Causes the following function to be generated from our template

```
double Min(double a, double b) {
  if (a < b)
      return a;
  else
      return b;
}
```

MIN TEMPLATE

- Code segment

```
complex r(6,21);
Complex s(11,29);
cout << Min(r, s) << endl;
```

- Causes the following function to be generated from our template

```
complex Min(complex a, complex b){
  if (a < b)
      return a;
  else
      return b;
}
```

Operator < needs to be defined for the actual template parameter type i.e. complex. If < is not defined, then a compile-time error occurs

FUNCTION TEMPLATE EXAMPLE 2: GENERIC SORTING

```
template <class T>
void InsertionSort(T A[], int n)
//T is data type of array A, n is size of A
{
  for (int i = 1; i < n; ++i) {
   if (A[i] < A[i-1]) {
      T val = A[i];
      int j = i;
      do {  A[j] = A[j-1];
         --j;
      } while ((j > 0) && (val < A[j-1]));
      A[j] = val;
   }
  }
}
```

FUNCTION TEMPLATES EXAMPLE 3

```
# include <iostream>
 using namespace std;
 template <class T>
T abs (T n)
{
  return (n<0) ? –n:n; }
 void main()
{
  int int1=5, int2=-6;
 long lon1=7000;
 double dub1=-10.15;
 cout<<abs(int1);  cout<<abs(lon1);  ….etc. }
```

MORE THAN ONE TEMPLATE ARGUMENT

```
 template <class atype, class btype>
// finds an element in an array
// atype is type of elements, btype is type of  size
 btype find (atype * array, atype value, btype size)
{
     for (btype j=0; j<size; j++)
          if (array[j] == value)
               return j;
     return (btype) (-1);
}
```

CLASS TEMPLATES

```
#include <iostream>
using namespace std;
template <class T>
class array {
T a[10];
int currentpos;
public:
array() {currentpos=-1;}
void insert (T var) {
a[++currentpos]=var;}
```

```
T remove() {
return a[currentpos--];}};
void main()
{
 array <float> arr1;
 array <int> arr2;
 arr1.insert(12.9);
cout<<arr1.remove();
arr2.insert(1234);
cout<<arr2.remove();}
```

Class Templates: Example 2

```
const int MAX=10;
template<class TYPE>
class stack
{
private:
    TYPE st[MAX]; //definition of st
    int top; //being an index always an int
public:
    stack()
    { top=-1;    }
    void push (TYPE n);
    TYPE pop();
};
```

```
template<class TYPE>
void stack<TYPE>::push(TYPE n)       //parameter to push
      { st[++top]=n; }
template<class TYPE>
TYPE stack<TYPE>::pop()              //return type of pop
      { return st[top--];}
int main()
{     stack<int> s1;
      s1.push(13) ; s1.push(22);
      cout<<s1.pop()<<endl;          //prints 22
      cout<<s1.pop()<<endl;          //prints 13
      stack<char> s2;
      s2.push('p') ; s2.push('6');
      cout<<s2.pop()<<endl;          //prints 6
      cout<<s2.pop()<<endl;          //prints p
}
```

CLASS WORK

Write a template function that returns the average of all the elements of an array with a generic data type. The arguments to the function should be the array name and the size of the array. In main () , exercise the function with arrays of type int, long, and double.

What are dangling pointers?

A GLIMPSE FROM PREVIOUS LECTURE

- StaticIntPointer Task
  - Error Solution
  - Avoid Memory Leakage
- Destructor